

# 25 — Rate-Limit Retry with Exponential Backoff

Doc 11 introduced backoff briefly in the embedding context (3 retries, starting at 2 seconds). This doc goes deeper on the LLM provider retry logic in `bilingual_etl/providers/llm_provider.py`, which uses much more aggressive settings — 6 retries starting at 30 seconds — and explains why naive retries cause a well-known failure pattern called the thundering herd.

## What it is

When our ETL pipeline calls an LLM API (for translation or enrichment), the API may reject the request with HTTP status code 429 ("Too Many Requests") because we've exceeded our rate limit. The retry-with-backoff pattern handles this by:

1. Catching the error.
2. Checking if the API's response includes a `Retry-After` header telling us exactly how long to wait.
3. If it does, waiting that long. If it doesn't, waiting for an exponentially growing duration (30s, then 60s, then 120s, then 240s...).
4. Trying again.
5. After 6 total attempts, giving up and raising an error.

The "exponential" part means each wait is double the previous one. This is the standard **exponential backoff** pattern used across the industry wherever software talks to a rate-limited API.

## Real-life analogy

You're calling a popular restaurant to make a reservation. The line is busy.

**Naive retry (no backoff):** You immediately redial. Busy. Redial. Busy. Redial. Busy. You're one of 50 people doing this simultaneously, and every one of you is making the line more busy by constantly calling. The restaurant's phone system is now overwhelmed not just by genuine callers but by retry callers — this is the **thundering herd**.

**Exponential backoff:** First busy signal — you wait 30 seconds and try again. Still busy — you wait a full minute. Still busy — you wait 2 minutes. Each time you back off longer, giving the phone line room to clear. By the time you try your fourth call, many of those 50 people have either gotten through or given up, so the line is less congested.

**Retry-After header:** Sometimes the restaurant's voicemail says "Please call back in 5 minutes." That's a `Retry-After` header — the server is telling you exactly when to try again instead of making you guess. When the server gives you this information, you trust it over your own backoff calculation because the server knows its own capacity better than you do.

## Why we used it here

This project uses free-tier LLM providers (Gemini, Groq, OpenRouter), and free tiers have strict rate limits — often as low as 15-30 requests per minute. The ETL pipeline processes thousands of records, each needing multiple LLM calls (translation + enrichment). At peak throughput, we absolutely will hit rate limits.

The retry settings in `llm_provider.py` are more aggressive than the embedding retry settings in `embeddings.py`:

| Setting | Embedding<br>retry | LLM<br>retry | Why the difference |
|---------|--------------------|--------------|--------------------|
|         | 3                  | 6            |                    |

| Setting            | Embedding<br>retry | LLM<br>retry  | Why the difference                                                                                                |
|--------------------|--------------------|---------------|-------------------------------------------------------------------------------------------------------------------|
| Max<br>retries     |                    |               | LLM rate limits have longer<br>cooldown windows; 3 tries isn't<br>enough to outlast a minute-long rate<br>window  |
| Initial<br>backoff | 2 seconds          | 30<br>seconds | Embedding rate limits tend to clear<br>in seconds; LLM free-tier rate limits<br>often need 30-60 seconds to reset |

The 30-second initial backoff is not a random number — it's calibrated to the observed behavior of free-tier LLM APIs. A per-minute rate limit (e.g., "15 requests per minute on Gemini free tier") means that after hitting the limit, you need to wait until the current minute window expires. Starting at 2 seconds (like the embedding path) would mean you'd retry 3 times in 6 seconds, fail all of them, and give up — when waiting 30 seconds would have succeeded. Starting at 30 seconds means your first retry has a real chance of landing in the next rate window.

## Code walkthrough

From `bilingual_etl/providers/llm_provider.py`:

### The configuration constants:

```
GENERATE_MAX_RETRIES = 6
GENERATE_INITIAL_BACKOFF_SECONDS = 30
```

6 retries starting at 30 seconds, doubling each time: 30, 60, 120, 240, 480 seconds. The total maximum wait before giving up is  $30 + 60 + 120 + 240 + 480 = 930$  seconds (about 15.5 minutes). That sounds long, but consider that a failed record after exhausting retries means that record is skipped entirely — the pipeline moves on, but that company or category has no translation. It's worth waiting 15 minutes for a rate limit to clear rather than losing data.

## Parsing the Retry-After header:

```
def _get_retry_after(error: Exception) -> float | None:
    resp = getattr(error, "response", None)
    if resp is None or resp.status_code != 429:
        return None
    retry_after = resp.headers.get("Retry-After") or resp.headers.get("retry-after")
    if retry_after:
        try:
            return max(float(retry_after), 1.0)
        except ValueError:
            pass
    return None
```

Walking through this:

1. `getattr(error, "response", None)` — the `requests` library attaches the HTTP response object to its exceptions. If the error isn't an HTTP error (e.g., a network timeout), there's no response, so we return `None`.
2. `resp.status_code != 429` — we only look for `Retry-After` on 429 (Too Many Requests) responses. A 500 (server error) or 503 (service unavailable) might also benefit from a retry, but those don't have a `Retry-After` header — we'll use our own backoff timing for those.
3. `resp.headers.get("Retry-After")` — HTTP headers can be case-insensitive, so we check both capitalizations. The header's value is the number of seconds the server wants us to wait.
4. `max(float(retry_after), 1.0)` — we trust the server's number but enforce a minimum of 1 second. A server saying "retry after 0 seconds" would be effectively "retry immediately," which defeats the purpose.
5. If parsing fails (the header value isn't a number), we return `None` and fall back to our own backoff.

## The retry loop:

```
def generate_with_retry(llm: LLMPProvider, prompt: str, system_prompt: str = "") -> str:
    backoff = GENERATE_INITIAL_BACKOFF_SECONDS
    last_error: Exception | None = None
```

```

for attempt in range(1, GENERATE_MAX_RETRIES + 1):
    try:
        return llm.generate(prompt, system_prompt=system_prompt)
    except Exception as e:
        last_error = e
        retry_after = _get_retry_after(e)
        wait = retry_after if retry_after else backoff
        logger.warning(
            f"LLM generate failed (attempt {attempt}/{GENERATE_MAX_RETRIES}):"
            + (f" | Retry-After: {retry_after}s" if retry_after else "")
        )
        if attempt < GENERATE_MAX_RETRIES:
            time.sleep(wait)
            backoff *= 2

raise RuntimeError(
    f"LLM generate failed after {GENERATE_MAX_RETRIES} attempts: {last_error}"
) from last_error

```

The decision tree on each failure:

1. Did the error include a 429 response with a `Retry-After` header? Use the server's wait time.
2. No `Retry-After`? Use our own exponential backoff (30s, 60s, 120s...).
3. Regardless of which wait time we used, double the backoff variable for next time. This means if the server's `Retry-After` was used on attempt 2, but attempt 3 doesn't have a `Retry-After`, the backoff will be at 120s (doubled twice from 30), not reset to 30.
4. On the last attempt, skip the sleep — there's no next attempt to wait for.
5. If all attempts fail, raise a `RuntimeError` that chains the original exception (via `from last_error`), so the caller sees both "retries exhausted" and the underlying API error.

### Why the thundering herd matters here:

When running with `--workers 4` (see doc 23), four threads are calling the LLM API simultaneously. If all four hit a rate limit at the same

moment and all retry immediately, you get 4 simultaneous retries — which are also rate-limited, so they all fail, and all retry again, creating a cascade of wasted requests. Exponential backoff with some natural variation (each thread started at a slightly different time, so their backoff timers are slightly offset) spreads the retries out, giving the rate limit window a chance to reset before the next wave of requests.

## **How to explain this in an interview**

"We use exponential backoff with Retry-After header parsing for all LLM API calls. When a call fails, we first check if the server sent a `Retry-After` header telling us exactly how long to wait — if it did, we trust that. If not, we use our own doubling backoff starting at 30 seconds, calibrated to the minute-long rate windows of free-tier LLM providers. The exponential growth prevents the thundering herd problem: if multiple threads hit a rate limit simultaneously and all retried instantly, they'd create a storm of requests that makes congestion worse. By spacing retries with growing delays, each retry has a real chance of landing in a cleared rate window. We allow up to 6 retries (about 15 minutes of total wait) because losing a record entirely is worse than waiting for a rate limit to clear."